# Holds and pinned coins

An extension to the booking model in State management: one coin can back several offers handed to a third party, of which at most one settles.

# Use cases

Each example: several offers from the same coins, handed to a third party that settles at most one.

| Example | Why pinning is needed |
|---|---|
| Bids on several equal lots | One bid per lot from the same funds. Today the second bid fails, and a transfer could spend the coin and void every bid. |
| Commitments to several token launches | The same NIGHT is offered to every launch; the coin must stay unspent until the first launch to reach its goal takes it. |
| Quotes: RFQ, offer files (MIP-0005), P2P swaps (MIP-0006) | A quote stays public until someone takes it; its coin must not be spent elsewhere, and the wallet must recognise the fill. |
| Pre-authorised charge for a metered station (having a collateral) | Funds are held as collateral up to a ceiling before the final amount is known; the rest of the balance stays spendable. |

**Current behaviour:** booking reserves a coin for one wallet-submitted transaction. A second build fails; discarding the first forgets an offer that can still be submitted.

# Required wallet guarantees

| Guarantee | Requirement |
|---|---|
| Coin reuse | Several offers may spend the same coins; managed by the user or by a dApp. |
| Coin holding | Coins can be reserved before any offer exists; released at an expiry. |
| Managed selection isolation | Ordinary selection never takes a held coin, and a dApp uses only its own holds. Only the user can act across dApps. |
| Settlement detection | When one offer lands: coin Spent, that offer Confirmed, the others invalid. Uses the existing nullifier matching. |
| Outstanding tracking | A dropped offer that already left the wallet stays on record until expiry. |
| Scope privacy | Only the owner of a hold (the user, or the dApp's origin) can see it. Other dApps cannot tell a held coin from a spent one. |
| Scope management | Each scope can list and manage what it created: its tag ids, offers and reservations. |
| Optional (opt-in) | dApps can skip it by not reading the booked status; wallets can opt out by not sending use tags. Without use tags, behaviour is unchanged. |
| Spec compatibility | An extension of the specification's booking: no new coin state, no new transaction status, nothing on chain. |

# Proposed API

| | |
|---|---|
| `spend()` | Opt-out: no tag, so coin selection works as today and never takes a booked coin. |
| `spend({ use: "auction-7" })` | Build the offer from coins already held for "auction-7", adding free coins if needed; every coin used joins the hold. |
| `spend({ use: "auction-7", mark: "auction-8" })` | Same coins as "auction-7", now also held for "auction-8": the next round reuses this round's coins. |
| `hold("session-1", amount, { expires_at })` | Reserve coins without building an offer yet; released at expires_at. |
| `getPins({ state: ['booked'] })` | Caller's scope only: tag ids (e.g. "auction-7") and their transactions, by state: 'booked' (default) or 'outstanding'. Never the coins. |
| `detach_transaction(offer \| { id }, { force? })` | Drop one offer or a whole hold and release its coins. The offer stays on record until it expires; force skips the record. |

- A hold belongs to the user or to one dApp (its origin); other dApps cannot see or use it.
- A wallet that cannot honour use must fail the build, not silently pick other coins.

# Coin pinned metadata

A Booked coin keeps one pin record per offer or hold. A scope reads only its own records, through getPins.




| tag-id | scope | expires_at | transaction-id |
|---|---|---|---|
| "launches" | https://market-a.example | 2026-10-14 09:00 UTC | 0x3f2a…c1a1 (offer A) |
| "launches" | https://market-a.example | 2026-10-14 09:05 UTC | 0x9c71…0b42 (offer B) |

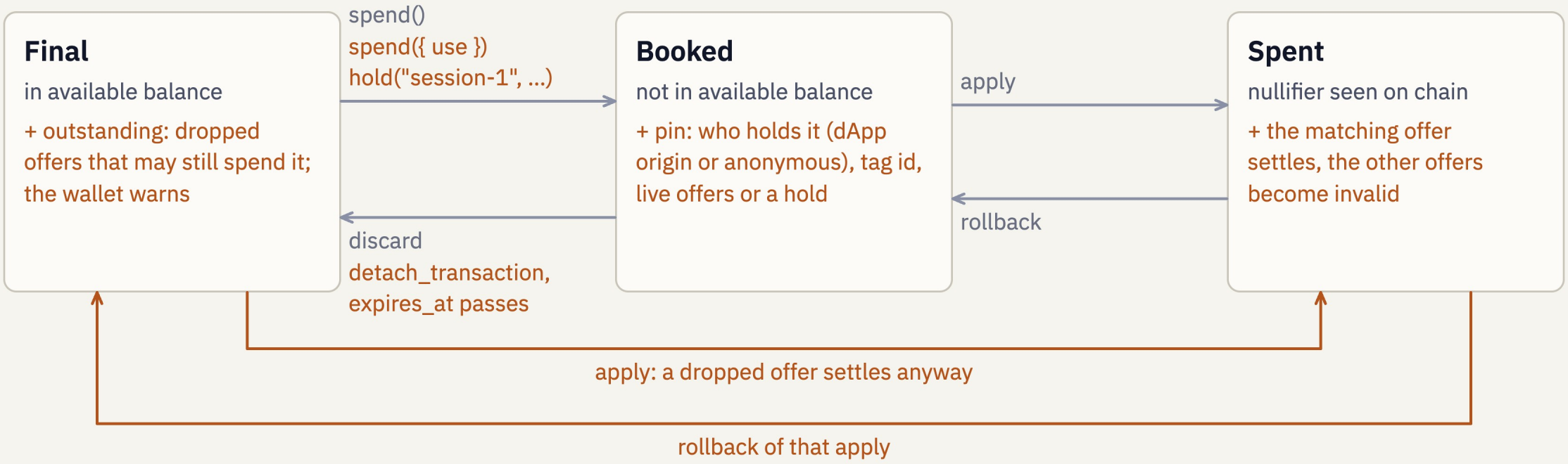



| tag-id | scope | expires_at | transaction-id |
|---|---|---|---|
| "session-1" | https://charger.example | 2026-09-30 18:00 UTC | none (hold, no offer yet) |
| "round-2" | anonymous | 2026-10-14 10:00 UTC | 0x71e0…9d0d |

Other scopes see neither record: to them both coins look spent.

# Coin state machine changes

Boxes are coin states. Arrows are transitions, labelled with the operation that causes them. Orange: added by this proposal. Pending and Confirmed are unchanged and not drawn.




**Transactions:** Pending gains submitter (wallet or dApp) · Rejected gains a reason · new edge Rejected → Confirmed